

Name -Mohit Gujarati

Subject - Applicable Deep Learning (AI700-001)

Student Id-100862494

A Comparative Survey of PyTorch vs. TensorFlow for Deep Learning: Usability, Performance, and Deployment

1. Scope of the Paper (Abstract)

This paper represents a comparative survey of TensorFlow and PyTorch. These frameworks are highly dominant in the field of deep learning in research and industry use. These two frameworks have been examined against each other based on four major categories, which include developer experience and coding styles, training and inference efficiency, deployment capabilities, and overall ecosystem. Having used a range of peer-reviewed metrics and industry whitepapers, the author states that both these platforms are equally capable of delivering good outcomes; however, each of them has certain strengths and weaknesses, with PyTorch being focused on simplicity and research, and TensorFlow being focused on completeness of a production-ready ecosystem. The proper choice depends on where the trade-offs can matter most.

2. Introduction

In 2015, Google developed TensorFlow; in the following year, Facebook released PyTorch, both of which emerged from a crowded field of deep learning frameworks to dominate both academia and industry. They had a common foundation of tensor computation, GPU acceleration, and automatic differentiation. TensorFlow originally used a static computational graph model, in which the full computation was defined and then executed within a session. On the other hand, PyTorch introduced a dynamic, define-by-run approach: every forward pass builds the graph on the fly, making the code behave like normal Python. By 2023, according to reports, PyTorch appeared in roughly 80% of NeurIPS papers that specified a framework, while TensorFlow had approximately 55% industry adoption compared to PyTorch's 20% in production environments. That split is what this paper sets out to explain, surveying not just raw performance but the broader question of which tool fits which context best.

3. Related Work

Comparative studies on the performance of deep learning frameworks began around the middle of the decade, starting in the 2010s. Bahrampour et al. (2015) compared three frameworks (Caffe, Theano, and Torch) within computer vision-related tasks and pointed out that the variation in CUDA use could lead to significant differences in efficiency. While Shi et al. (2016) evaluated the best options for this period of time, many frameworks mentioned therein (including Theano and Torch7) lost popularity since then. More relevant and timely is the study by Yapici and Topaloğlu (2021), revealing that TensorFlow was only slightly faster when processing small image datasets (MNIST) while demonstrating inferior performance to PyTorch when the image size grew.

Similarly, in their experiments related to convolutional neural networks, Novac et al. (2022) found that PyTorch performed about 25.5% better in training time while offering approximately 77% faster performance during inference. Industry polls, such as the one published by Stack Overflow (2023 Developer Survey) and conducted by Kaggle practitioners, show that TensorFlow performs better in production settings, while PyTorch is used more often for research purposes and prototyping. This particular study is distinctive from other sources in focusing on TensorFlow 2.x as opposed to older versions, as well as in using more current statistics.

4. Code Style and Developer Experience

The key distinguishing factor of PyTorch is its dynamic nature. The operations are evaluated right away when the code executes, hence making debugging easy because regular debuggers can be used, and the error would clearly show where the problem arises. In contrast, TensorFlow 1.x had users define the computational graph first before evaluating it using the `tf.Session` method. This means separation between the modeling part and the execution part, and can appear to contradict the typical flow of building a program in Python. In TensorFlow 2.x, eager execution has been introduced, whereby the flow of programming in TF becomes similar to that of PyTorch, although graph optimization remains possible with the `@tf.function` decorator.

In both frameworks, an object-oriented approach is adopted in terms of model building. While PyTorch uses `torch.nn.Module` along with the explicit `forward()` method to have control over every training cycle, TensorFlow uses the `model.compile()` and `model.fit()` provided through the Keras interface. These two methods simplify training by combining everything into one, though at the cost of having little room for customization compared to the traditional way of constructing the training loop. In empirical studies, novices usually find PyTorch more Python-like, whereas seasoned developers usually prefer TensorFlow due to its comprehensive support. A fitting example from the given source compares PyTorch with a scripting language and TensorFlow with an app framework.

5. Performance Comparison (Training & Inference)

With respect to efficiency in throughput during training procedures, PyTorch is proven to have some advantages over TensorFlow, particularly in single-GPU training of models of medium or larger sizes. Specifically, for stereo-vision convolutional neural networks, PyTorch recorded 16.98 hours (or approximately 1.18 hours per epoch), which is equivalent to approximately a 25.5% improvement relative to TensorFlow's 21.95 hours (1.53 hours per epoch) according to Novac et al. (2022). However, regarding image size-dependent variations, Yapıcı & Topaloğlu (2021) found that TensorFlow has faster processing times for small-sized data sets such as MNIST, which may be attributed to less overhead, but PyTorch has faster speeds than TensorFlow as images become bigger in resolution. Concerning inference, the disparity becomes even more pronounced. According to Bećirović et al. (2025), PyTorch processes 28×28 images in 0.303 seconds while TensorFlow needs 0.898 seconds to do the same job, giving PyTorch a nearly three-fold advantage. The margin becomes smaller with larger input sizes, with PyTorch taking 1.596 seconds and TensorFlow 2.018 seconds to process 64×64 images. Besides that, PyTorch's use of a cache allocator makes its GPU memory allocation more effective as compared to TensorFlow's upfront allocation.

At scale, TensorFlow has traditionally held advantages through its mature TPU support (XLA compiler) and established distributed training tools (MirroredStrategy, TPUStrategy). PyTorch's Distributed Data Parallel has largely closed that gap for multi-GPU training, scaling linearly in many configurations. MLPerf benchmark results place both frameworks in the same competitive ballpark, with differences more attributable to model-tuning choices than to core framework efficiency. PyTorch 2.0's torch. A compile mechanism that JIT compiles code without abandoning eager semantics has further narrowed the performance gap with statically compiled graphs.

6. Applications and Case Studies in Vision, NLP, Etc.

In computer vision, both frameworks have deep roots. Google's earlier efforts, like DeepLab and Inception, have been built on TensorFlow, and TensorFlow's Object Detection API is still extensively used. Meta countered with Detectron2 in PyTorch, and researchers have gravitated toward PyTorch for newer architectures, GANs, Vision Transformers, and diffusion models, which almost always arrive with PyTorch reference implementations. Tesla uses PyTorch for Autopilot perception models; Toyota Research for autonomous driving vision. In the field of natural language processing (NLP), the move towards PyTorch has gained momentum: While BERT was built on TensorFlow, GPT (by OpenAI) moved away from TensorFlow from the get-go, and Hugging Face Transformers started as PyTorch-only models, Microsoft too switched to PyTorch in its NLP research for greater flexibility. TensorFlow retains scale in production NLP Google Translate runs on TF over TPU pods. In recommender systems, Meta's DLRM is PyTorch-based while Google's RecNet uses TensorFlow. Notable industry examples involve training OpenAI's GPT-3 on large GPU clusters using PyTorch, Airbnb's customer service chatbot trained using PyTorch, Genentech's cancer drug development pipeline that leverages PyTorch, and Snapchat's mobile ML capabilities powered by TensorFlow Lite.

7. Concluding Remarks

The conclusion of this work refutes any direct comparison of these two approaches. TensorFlow comes out on top for those who prioritize deployment scalability thanks to its ecosystem, which consists of TFLite, TF.js, TF Serving, and TFX, which provides a full end-to-end solution for training and deployment across platforms, which is only partly implemented in PyTorch without additional effort. However, PyTorch is still superior for research projects and rapid development. With tools like TorchScript, ONNX, and TorchServe, PyTorch catches up in terms of deployment capabilities. For future work, convergence is expected. On one hand, PyTorch 2.0 is set to introduce torch. Compile for hybrid eager and graph mode, while on the other hand, the TensorFlow team is already integrating features borrowed from JAX that allow it to implement hybrid execution. As a result, the choice of framework can no longer be based solely on technical aspects but also depends on the ecosystem, deployment target, and skills of the development team.

References

- **Ba Alawi, Z. (2025).** A Comparative Survey of PyTorch vs TensorFlow for Deep Learning: Usability, Performance, and Deployment Trade-offs. <https://arxiv.org/abs/2508.04035>
- **Abadi, M., et al. (2016).** TensorFlow: Large-scale machine learning on heterogeneous distributed systems. <https://arxiv.org/abs/1603.04467>
- **Bahrampour, S., et al. (2015).** Comparative study of Caffe, Neon, Theano, and Torch for deep learning. <https://arxiv.org/abs/1511.06435>
- **Bećirović, M., et al. (2025).** Performance comparison of medical image classification systems using TensorFlow Keras, PyTorch, and JAX. <https://arxiv.org/abs/2507.14587>
- **Nandan, A. D. M. (2024).** Comparative analysis of TensorFlow and PyTorch for image classification using CNN with parallelism strategies. In 2024 IEEE Silchar Subsection Conference (SILCON 2024) (pp. 1-6). IEEE. <https://ieeexplore.ieee.org/document/10910577>
- **Yapici & Topaloglu (2021):** This research illustrates specific performance trade-offs, showing TensorFlow is slightly faster for small images, while PyTorch is faster and manages memory better for larger images. <https://dergipark.org.tr/en/download/article-file/1201877>
- **Novac, O.-C., Chirodea, M. C., Novac, C. M., Bizon, N., Oproescu, M., Stan, O. P., & Gordan, C. E. (2022).** Analysis of the application efficiency of TensorFlow and PyTorch in a convolutional neural network. Sensors, 22(22), 8872. <https://www.mdpi.com/1424-8220/22/22/8872>
- **Paszke, A., et al. (2019).** PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems. https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf